



Data: 22/01/2025

Progetto: OikoNaos	Versione: 1.7
Documento: Object Design	Data: 22/01/2025

Coordinatore del progetto:

Nome	Matricola
Luigi Potestà	0512120076

Partecipanti:

Nome	Matricola
Luigi Potestà	0512120076
Maria Antonietta Ruggiero	0512121126
Giulia Buonafine	0512119695
Mario Sinopoli	0512119113

Scritto da:	Giulia Buonafine
--------------------	------------------

Revision History

Data	Versione	Descrizione	Autore
01/12/2025	1.0	Prima versione del documento.	Giulia Buonafine
02/12/2025	1.1	Revisione completa del Purpose, Packages e Class Interfaces Glossary; aggiunta sottosistema Notifiche.	Luigi Potestà
03/12/2025	1.2	Correzione coerenza interna e allineamento con RAD 3.10 e SDD 1.6.	Maria Antonietta Ruggiero
04/12/2025	1.3	Rifinitura formattazione, correzioni linguistiche minori e miglioramento consistenza terminologica; aggiunto diagramma UML dei Package.	Mario Sinopoli
07/12/2025	1.4	Ridefinizione package e Class Interfaces Glossary	Giulia Buonafine
12/12/2025	1.5	Adattamento dei packages alle funzionalità implementate	Luigi Potestà Maria Antonietta Ruggiero
18/12/2025	1.6	Ridefinizione Class Interfaces Glossary adattandola alla versione 1.7 del file System Design	Giulia Buonafine Mario Sinopoli

22/01/2026	1.7	Refactoring architetturale completo. Aggiunta sottosistemi Gestione Finanziaria, DTO, Filtri di Sicurezza e aggiornamento DAO.	Maria Antonietta Ruggiero
-------------------	------------	--	---------------------------

Indice

1. Purpose	4
2. Audience.....	4
3. Introduction	5
3.1. Object design trade-offs	5
3.2. Interface documentation guidelines	6
3.3. Definitions, Acronyms and Abbreviations	7
3.4. References	8
4. Packages	8
5. Class Interfaces Glossary	10

1. Purpose

Il presente Object Design Document ha lo scopo di descrivere in modo dettagliato la struttura interna del sistema OikoNaos, con particolare riferimento alle classi, ai package e alle relazioni tra i componenti software che costituiscono il prototipo della piattaforma. Il documento fornisce una visione approfondita dell'implementazione orientata agli oggetti del sistema, fungendo da collegamento tra le specifiche funzionali definite nel Requirements Analysis Document e le scelte architetturali illustrate nel System Design Document.

L'ODD è focalizzato sulla descrizione dei componenti effettivamente implementati nel prototipo iniziale di OikoNaos, che include un insieme rappresentativo di funzionalità fondamentali per la gestione delle comunità di co-housing. In particolare, il prototipo comprende i moduli di autenticazione e controllo degli accessi, la gestione delle prenotazioni degli spazi comuni, la creazione e consultazione dei ticket di manutenzione, la gestione delle risorse condivise, la gestione e partecipazione agli eventi della comunità e il modulo di gestione delle tasse e dei pagamenti. Per quanto riguarda la gestione delle credenziali, il perimetro implementativo è limitato alla funzionalità di reset della password, in conformità con quanto previsto per la versione prototipale del sistema.

Il documento descrive l'organizzazione dei package secondo il pattern architetturale Model–View–Controller, adottato per garantire separazione delle responsabilità, manutenibilità del codice e coerenza con l'architettura multilivello definita nello SDD. In tale contesto, vengono analizzate le classi appartenenti al livello di presentazione (controller e filtri), al livello applicativo e al livello di persistenza (entità e Data Access Object), evidenziandone ruoli, responsabilità e principali interazioni.

L'obiettivo del presente ODD non è quello di coprire l'intera piattaforma OikoNaos nella sua visione finale, ma di documentare in modo accurato e coerente la struttura del prototipo effettivamente sviluppato e sottoposto ad attività di verifica e validazione. Il documento costituisce pertanto un riferimento tecnico per il team di sviluppo e di testing, supportando le attività di manutenzione, estensione futura del sistema e tracciabilità tra requisiti, progettazione e implementazione.

2. Audience

Il documento di Object Design Document è rivolto alle figure tecniche coinvolte nello sviluppo, nella verifica e nell'evoluzione della piattaforma OikoNaos, fornendo una descrizione chiara e strutturata delle scelte progettuali adottate a livello di progettazione a oggetti.

- **System Architects:** Utilizzano l'ODD per verificare che la progettazione dei componenti software sia coerente con l'architettura multilivello definita nel System Design Document (SDD) e con i principi di progettazione oggetto del corso, quali elevata coesione, basso accoppiamento e corretta separazione delle responsabilità.
- **Sviluppatori:** L'ODD rappresenta un riferimento operativo per l'implementazione del codice sorgente. Il documento descrive le classi, le loro responsabilità, le dipendenze ammesse e le interazioni tra i componenti, supportando uno sviluppo coerente e allineato alle scelte architetturali.
- **Tester:** Il documento fornisce ai tester le informazioni necessarie per comprendere il comportamento atteso dei componenti software, individuare flussi critici, condizioni di errore ed eccezioni rilevanti, e progettare attività di verifica coerenti con la struttura interna del sistema.

- **Stakeholder dell'Azienda Coordinatrice:** Pur non essendo direttamente coinvolti nelle attività di sviluppo, gli stakeholder possono consultare l'ODD a scopo informativo per comprendere le principali scelte progettuali adottate, con particolare riferimento agli aspetti di sicurezza, gestione dei dati e controllo degli accessi basato sui ruoli (RBAC).

3. Introduction

L'Object Design rappresenta la fase del processo di sviluppo in cui il modello concettuale definito durante l'analisi viene progressivamente trasformato in una struttura software concreta.

In questa fase, i concetti astratti del dominio applicativo vengono mappati in classi, metodi e responsabilità ben definite, introducendo decisioni tecniche e soluzioni progettuali necessarie alla realizzazione del sistema.

Il modello viene arricchito attraverso pattern ricorrenti e componenti appartenenti al dominio della soluzione, come i Data Access Object (DAO) e le classi di supporto alla persistenza dei dati, che consentono di separare la logica applicativa dai dettagli implementativi.

Nel contesto di OikoNaos, l'Object Design è focalizzato sui sottosistemi effettivamente implementati nella versione prototipale iniziale del sistema. In particolare, la progettazione a oggetti riguarda le funzionalità di autenticazione degli utenti, la gestione delle prenotazioni degli spazi comuni e la creazione e consultazione dei ticket di manutenzione. Le funzionalità relative alla gestione avanzata del profilo utente e agli altri sottosistemi previsti dall'architettura complessiva sono considerate parte di evoluzioni future e non rientrano nel perimetro implementativo del prototipo.

Per ciascun sottosistema incluso nel prototipo vengono presentati gli oggetti di tipo Boundary, Control ed Entity, illustrandone il ruolo all'interno del modello e le principali responsabilità. L'organizzazione delle classi è guidata da principi cardine della progettazione orientata agli oggetti, quali la separazione delle responsabilità, l'uso della delega per ridurre l'accoppiamento, la distinzione tra oggetti del dominio applicativo e oggetti introdotti per esigenze implementative, e l'adozione di un'architettura chiusa che protegge i dettagli interni dei componenti.

3.1. Object design trade-offs

Nel processo di progettazione a oggetti della piattaforma OikoNaos sono state effettuate diverse scelte progettuali che comportano compromessi tra obiettivi contrastanti. La presente sezione descrive i principali trade-off considerati, evidenziando le motivazioni alla base delle decisioni adottate nella versione prototipale del sistema.

Sicurezza vs. semplicità di implementazione: Per garantire un livello adeguato di sicurezza nell'accesso alla piattaforma, il sistema adotta meccanismi di autenticazione basati sulla protezione delle credenziali degli utenti, come l'hashing delle password e la gestione controllata delle sessioni. Tali scelte aumentano la complessità dell'implementazione rispetto a soluzioni più semplici, ma risultano necessarie per assicurare la protezione dei dati personali anche nella versione prototipale del sistema.

Riutilizzo di componenti off-the-shelf vs. controllo interno: Nella realizzazione del prototipo sono stati utilizzati componenti e librerie standard per la gestione dell'interfaccia web e di funzionalità di

supporto, riducendo il livello di controllo diretto su alcune parti del sistema. Questa scelta consente di concentrare lo sforzo progettuale e implementativo sulle funzionalità centrali del prototipo, quali autenticazione, prenotazioni e ticket di manutenzione, abbreviando i tempi di sviluppo e aumentando la stabilità complessiva.

Manutenibilità del codice vs. tempo di sviluppo: Nonostante i vincoli temporali, la progettazione del sistema segue il paradigma Model–View–Controller e una chiara separazione tra classi Entity, Control e componenti di accesso ai dati (DAO). Tale approccio comporta un incremento del numero di classi e una maggiore articolazione del design, ma migliora sensibilmente la leggibilità del codice, la manutenibilità e la possibilità di estendere il sistema in fasi successive.

Coerenza del modello dati vs. complessità della persistenza: La scelta di modellare entità distinte e ben definite, come Utente, Prenotazione e Ticket di Manutenzione, comporta una struttura del database più articolata, con un numero maggiore di tabelle e relazioni. Tuttavia, questa decisione favorisce la coerenza dei dati, l'integrità referenziale e una rappresentazione chiara dei concetti del dominio, riducendo il rischio di inconsistenze nelle operazioni di prenotazione e segnalazione.

Prestazioni vs. controlli di integrità nelle prenotazioni: Per evitare situazioni di conflitto, come prenotazioni sovrapposte degli spazi comuni, il sistema esegue controlli di coerenza che coinvolgono più entità del dominio prima di confermare una richiesta. Questo approccio può comportare un lieve aumento dei tempi di risposta, ma garantisce un comportamento affidabile e corretto del sistema, aspetto prioritario per una gestione equa delle risorse condivise.

3.2. Interface documentation guidelines

La documentazione delle interfacce ha l'obiettivo di garantire uniformità, chiarezza e leggibilità all'interno del progetto OikoNaos. A tal fine, la progettazione a oggetti e la relativa documentazione adottano convenzioni sintattiche e stilistiche coerenti, che consentono di identificare immediatamente il ruolo e la natura degli elementi descritti. Nel progetto OikoNaos vengono adottate due principali convenzioni di naming: PascalCase e camelCase, utilizzate in modo sistematico a seconda della tipologia dell'elemento.

La convenzione **PascalCase**, in cui ogni parola inizia con una lettera maiuscola senza l'uso di spazi o caratteri separatori, è utilizzata per tutti gli elementi che rappresentano concetti strutturali del sistema. In particolare, essa è adottata per le classi Entity, per le classi Control, per le classi Boundary e per i componenti di accesso ai dati (DAO). Esempi sono Utente, Prenotazione, Ticket, AutenticazioneControl, PrenotazioneControl, AutenticazioneView e UtenteDAO. L'uso di PascalCase consente di riconoscere immediatamente le componenti strutturali del sistema e garantisce un'elevata omogeneità formale all'interno del modello e del codice sorgente.

La notazione **camelCase**, in cui la prima parola è in minuscolo e le successive iniziano con una lettera maiuscola, è utilizzata per i nomi dei metodi, delle variabili di istanza e dei parametri. Esempi di metodi sono verificaCredenziali() e creaPrenotazione(), mentre esempi di variabili e parametri includono emailUtente, dataPrenotazione, idUtente e nuovaPassword. Questa distinzione visiva consente di differenziare chiaramente classi, metodi e attributi, migliorando la leggibilità sia del codice sia della

documentazione.

Per garantire coerenza interna nel documento e nell'implementazione, tutti gli identificatori devono rispettare le seguenti linee guida generali. I nomi devono riflettere chiaramente il ruolo della classe o del metodo; in particolare, i controller devono esplicitare l'azione o il processo che coordinano, mentre i metodi devono descrivere il comportamento che implementano. È necessario evitare abbreviazioni non standard, ad eccezione di quelle comunemente riconosciute come DAO, DB e ID. È inoltre preferibile adottare nomi descrittivi piuttosto che nomi eccessivamente brevi, al fine di rendere il codice autoesplicativo. I nomi riportati nella documentazione devono infine essere coerenti con quelli utilizzati nell'implementazione, evitando discrepanze tra progetto e codice.

Al fine di facilitare l'identificazione immediata del livello architetturale di appartenenza, vengono adottati suffissi convenzionali: le classi Boundary terminano con il suffisso *View*, i controller con *Control* e i componenti di persistenza con DAO. Questa scelta contribuisce a rendere evidente la separazione delle responsabilità all'interno dell'architettura Model–View–Controller.

Ogni classe descritta nell'Object Design Document deve essere accompagnata da una descrizione sintetica del proprio ruolo, dall'elenco delle principali responsabilità, dai metodi rilevanti con la relativa firma, dalle eventuali eccezioni sollevate e dalle collaborazioni con altre classi del sistema. Le interfacce devono essere definite in modo da riflettere chiaramente lo scopo della classe, senza entrare nei dettagli implementativi, che appartengono alla fase di codifica.

3.3. Definitions, Acronyms and Abbreviations

Termine	Definizione
Utente	Soggetto registrato al sistema OikoNaos che accede alla piattaforma tramite credenziali personali. Ogni utente è associato a un ruolo che ne determina i privilegi di accesso alle funzionalità del sistema.
Coinquilino	Utente con ruolo standard, autorizzato a utilizzare le funzionalità operative del sistema, tra cui la visualizzazione e prenotazione degli spazi comuni, la partecipazione agli eventi, l'invio di richieste di risorse condivise, la creazione di ticket di manutenzione e il pagamento delle tasse assegnate.
Supervisore	Utente con ruolo amministrativo responsabile della gestione della comunità. È autorizzato a creare e gestire eventi, risorse condivise e tasse, a supervisionare le prenotazioni, ad approvare o rifiutare richieste di risorse e a gestire i ticket di manutenzione.
Sessione Applicativa	Contesto di esecuzione associato a un utente autenticato, utilizzato per mantenere lo stato dell'interazione durante la navigazione dell'applicazione. La sessione contiene le informazioni essenziali sull'utente ed è utilizzata per il controllo degli accessi.
Filtro di Autenticazione	Componente del livello di presentazione incaricato di intercettare le richieste HTTP e verificare la presenza di una sessione valida. In assenza di autenticazione, il filtro impedisce l'accesso alle risorse protette e reindirizza l'utente alla pagina di login.
Evento	Attività sociale organizzata all'interno della comunità di co-housing. Un evento è caratterizzato da titolo, descrizione, data, luogo e numero massimo di partecipanti. Può essere creato e gestito dal Supervisore e prevede l'iscrizione e la disiscrizione dei Coinquilini nel rispetto dei vincoli di disponibilità.

Risorsa Condivisa	Bene o servizio messo a disposizione della comunità. Può essere richiesto dai Coinquilini previa verifica di disponibilità ed è soggetto ad approvazione o rifiuto da parte del Supervisore.
Prenotazione	Associazione temporale tra un utente e uno spazio comune, che consente l'uso esclusivo della risorsa in un determinato intervallo di tempo. Il sistema impedisce la creazione di prenotazioni in conflitto.
Ticket di Manutenzione	Segnalazione inviata da un Coinquilino per notificare problemi o guasti relativi agli spazi o alle risorse della comunità. Ogni ticket è associato a uno stato che ne rappresenta l'avanzamento ed è gestito dal Supervisore.
Tassa	Obbligazione economica assegnata ai Coinquilini dal Supervisore. È caratterizzata da importo, scadenza e stato di pagamento e può essere associata a uno o più pagamenti.
Pagamento	Operazione mediante la quale un Coinquilino salda una tassa assegnata. Il pagamento aggiorna lo stato della tassa e determina la generazione di una ricevuta.
Ricevuta	Documento generato dal sistema a seguito di un pagamento completato, contenente le informazioni riepilogative dell'operazione effettuata.
Reset della Password	Procedura che consente a un utente autenticato di modificare la propria password. Nel prototipo di OikoNaos, la gestione delle credenziali è limitata esclusivamente a questa funzionalità, nel rispetto dei requisiti di sicurezza.
DAO (Data Access Object)	Componente software responsabile dell'accesso ai dati persistiti nel database. I DAO incapsulano la logica di interazione con il DBMS e forniscono operazioni di creazione, lettura, aggiornamento e cancellazione delle informazioni.

3.4. References

- **Requirements Analysis_OikoNaos – Versione 3.11:** Documento che descrive i requisiti funzionali e non funzionali del sistema, utilizzato come riferimento principale per le scelte progettuali.
- **System Design_OikoNaos – Versione 1.8:** Documento che definisce l'architettura multilivello del sistema, la decomposizione in sottosistemi e i diagrammi UML che costituiscono la base per la progettazione a oggetti descritta nel presente ODD.

4. Packages

La progettazione a livello di oggetti del sistema OikoNaos è organizzata in package logici, ciascuno dei quali raggruppa classi con responsabilità omogenee. Questa suddivisione riflette direttamente l'architettura multilivello descritta nel System Design Document e supporta la separazione delle responsabilità prevista dal pattern Model–View–Controller (MVC).

L'organizzazione in package ha l'obiettivo di migliorare la leggibilità del codice, favorire la manutenibilità e semplificare le attività di testing e di evoluzione del sistema. Ogni package rappresenta un sottosistema funzionale coerente con i requisiti definiti nel Requirements Analysis Document.

Il package **controller** contiene le classi responsabili della gestione delle richieste HTTP e dell'implementazione dei flussi applicativi. Ogni controller coordina l'interazione tra il livello di presentazione e il livello di persistenza, applicando le regole di business previste per i casi d'uso inclusi nel prototipo. In questo package rientrano, tra gli altri, i controller per l'autenticazione degli utenti, la

gestione delle prenotazioni, delle risorse condivise, degli eventi, dei ticket di manutenzione e delle tasse e pagamenti.

Il package **model** raccoglie le classi che rappresentano le entità del dominio applicativo. Tali classi modellano i concetti fondamentali del sistema, come Utente, Prenotazione, Evento, Risorsa, Ticket, Tassa, Pagamento e Ricevuta, e contengono esclusivamente dati e metodi di accesso agli attributi, senza includere logica di controllo o di persistenza.

Il package **dao** include le classi Data Access Object, responsabili dell'interazione con il database relazionale MySQL. Ogni DAO incapsula le operazioni di creazione, lettura, aggiornamento e cancellazione delle entità del dominio, isolando la logica di persistenza dal resto dell'applicazione. Questa scelta progettuale consente di facilitare il testing dei componenti applicativi e ridurre l'accoppiamento tra i livelli del sistema.

Il package **filter** contiene i filtri applicativi utilizzati per il controllo dell'accesso alle risorse. In particolare, include il filtro di autenticazione, incaricato di verificare la presenza di una sessione valida prima di consentire l'accesso alle funzionalità protette. Tale meccanismo supporta l'applicazione delle politiche di sicurezza di base definite nel RAD ed è integrato nei flussi di autenticazione.

Il package **util** raccoglie classi di supporto utilizzate trasversalmente dai diversi moduli applicativi, come componenti per la gestione delle connessioni al database, la validazione dei dati e altre funzionalità di utilità generale. Queste classi non rappresentano concetti del dominio ma forniscono servizi comuni necessari al funzionamento del sistema.

Nel complesso, la struttura dei package adottata garantisce una chiara separazione delle responsabilità, un'elevata coerenza con l'architettura definita nel SDD e un allineamento diretto con i requisiti funzionali e non funzionali individuati nel RAD. Tale organizzazione costituisce una base solida per l'implementazione del prototipo e per le successive attività di verifica e manutenzione.

Di seguito viene dettagliata la struttura dei package e il loro contenuto.

Package	Descrizione e Contenuto
it.unisa.oikonaos.model	Model. Contiene le classi entity (JavaBean) che rappresentano lo stato del sistema e mappano le tabelle del database relazionale. Le classi di questo package modellano i concetti fondamentali del dominio applicativo (es. Utente, Prenotazione, Evento, Ticket, Tassa, Pagamento, Ricevuta) e incapsulano esclusivamente dati e logica di dominio di base, senza dipendenze dal livello di persistenza o di presentazione.
it.unisa.oikonaos.dao	Data Access Object (DAO). Gestisce l'accesso ai dati e isola la logica applicativa dai dettagli di persistenza su database MySQL. Ogni classe DAO implementa le operazioni CRUD e le query necessarie per una specifica entità del dominio, garantendo l'incapsulamento delle transazioni e delle interazioni JDBC.
it.unisa.oikonaos.controller	Controller. Contiene le Servlet Java responsabili della gestione delle richieste HTTP e dell'implementazione dei casi d'uso. Ogni controller coordina il flusso applicativo validando gli input, invocando i DAO necessari e determinando la vista di risposta. Questo package costituisce il punto di integrazione tra il livello di presentazione e il livello di persistenza.

it.unisa.oikonaos.dto	Data Transfer Objects (DTO). Contiene classi di supporto utilizzate per il trasferimento di dati strutturati o aggregati tra il controller e il livello di presentazione. I DTO consentono di disaccoppiare il modello persistente dalla view, migliorando la sicurezza e la manutenibilità dell'interfaccia utente.
it.unisa.oikonaos.filter	Security Layer. Contiene i filtri Servlet deputati al controllo dell'accesso alle risorse applicative. I filtri implementano le politiche di autenticazione e di controllo degli accessi basate sui ruoli (RBAC), garantendo che le funzionalità riservate siano accessibili esclusivamente agli utenti autorizzati.
it.unisa.oikonaos.util	Utilities. Raccoglie classi di supporto infrastrutturale utilizzate trasversalmente dal sistema, come la gestione delle connessioni al database e la validazione delle credenziali. Tali componenti forniscono servizi comuni senza appartenere al dominio applicativo.

5. Class Interfaces Glossary

Questa sezione descrive le principali classi coinvolte nella versione implementativa della piattaforma OikoNaos, così come definite nell'Object Design Document e rappresentate nei diagrammi architetturali di riferimento. Le classi sono organizzate secondo il pattern Model–View–Controller (MVC) adottato a livello di sistema, in coerenza con quanto specificato nel System Design Document.

La descrizione è strutturata evidenziando i quattro livelli principali dell'architettura: **View**, **Controller**, **Model** e **DAO**, mettendo in luce le responsabilità di ciascun componente, le collaborazioni rilevanti e il ruolo svolto nei flussi applicativi derivati dai casi d'uso del Requirements Analysis Document.

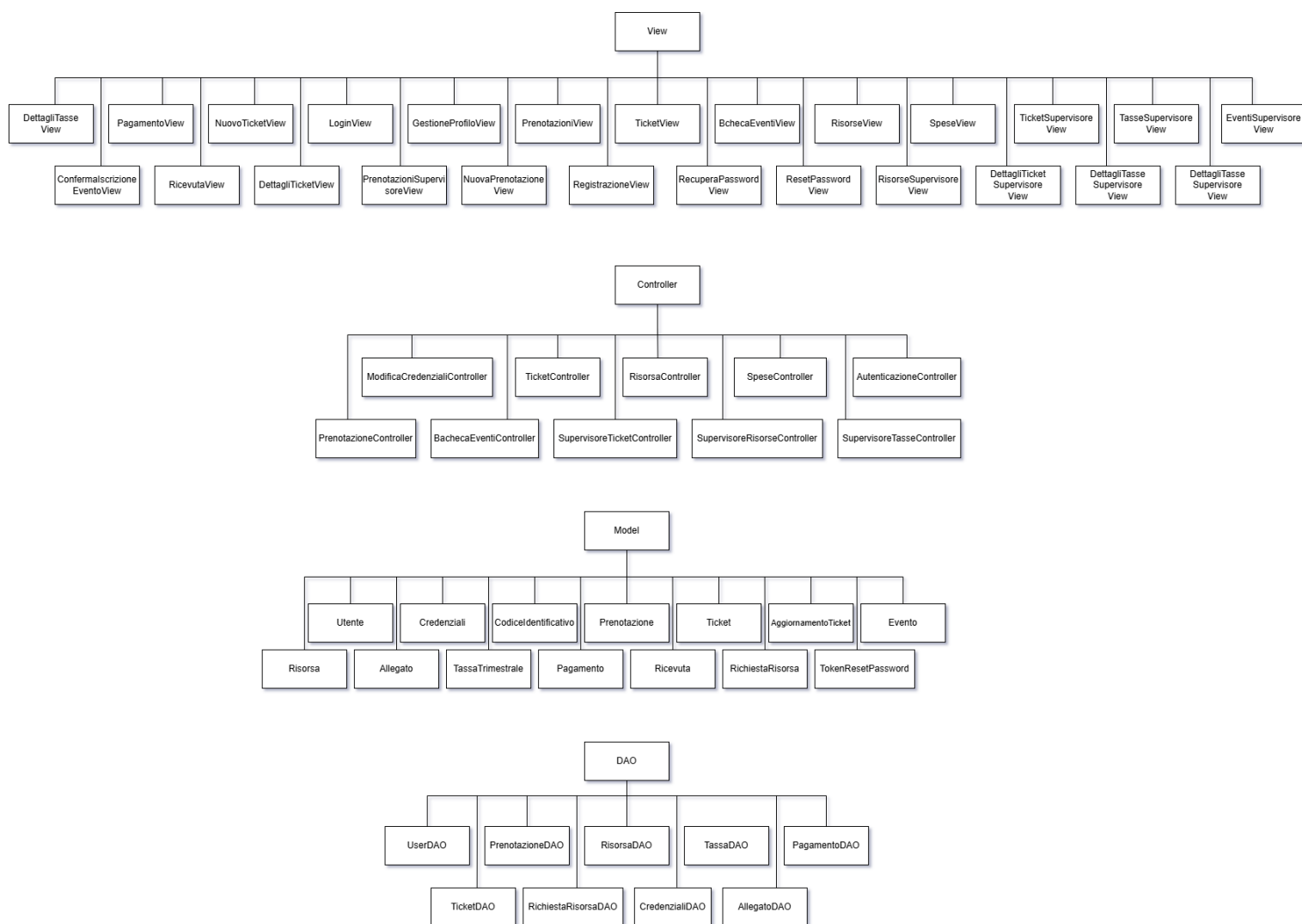
Il **livello di Presentazione (View)** comprende le classi responsabili della visualizzazione delle informazioni e dell'interazione con l'utente finale. Tali componenti rappresentano le interfacce dedicate ai diversi moduli funzionali del sistema, tra cui autenticazione, gestione del profilo, prenotazioni, ticket di manutenzione, eventi, risorse condivise e spese. Le View non contengono logica di business e dipendono esclusivamente dai controller per l'elaborazione delle richieste.

Il **livello di Applicazione (Controller)** include le Servlet che implementano i flussi operativi dei casi d'uso. Ogni controller coordina una specifica funzionalità, validando gli input dell'utente, invocando le operazioni sul modello tramite i DAO e selezionando la vista appropriata. Sono presenti controller distinti per i ruoli Coinquilino e Supervisore, in linea con il modello di controllo degli accessi definito nel RAD e nel SDD.

Il **livello di Dominio (Model)** rappresenta lo stato del sistema attraverso classi entity che mappano le principali entità persistenti, quali utenti, credenziali, prenotazioni, eventi, risorse, ticket, tasse, pagamenti e ricevute. Queste classi incapsulano i dati e, ove necessario, includono logica di dominio elementare (ad esempio la validità temporale dei token di reset password), senza dipendere dai livelli di presentazione o di persistenza.

Il **livello di Persistenza (DAO)** è responsabile dell'accesso ai dati e dell'interazione con il database relazionale MySQL. Ogni DAO incapsula le operazioni di creazione, lettura, aggiornamento e

cancellazione per una specifica entità del dominio, isolando la logica applicativa dai dettagli implementativi del database e favorendo la manutenibilità e la testabilità del sistema.



Gestione Autenticazione, Registrazione e Reset Password

Il modulo di Gestione Autenticazione e Credenziali è responsabile dell'identificazione sicura degli utenti e del controllo degli accessi alla piattaforma OikoNaos. Esso implementa i flussi di registrazione tramite Codice Identificativo Speciale, autenticazione, terminazione della sessione e recupero delle credenziali di accesso mediante meccanismo di reset password basato su token di sicurezza. Il modulo non include funzionalità di modifica estesa del profilo utente, in coerenza con quanto definito nel RAD e implementato nel prototipo.

Categoria	Classe	Descrizione
Boundary	LoginView	(JSP) Interfaccia di autenticazione dell'utente alla piattaforma
Boundary	RegistrazioneView	(JSP) Interfaccia per la registrazione tramite

		Codice Identificativo Speciale
Boundary	RecuperaPasswordView	(JSP) Interfaccia per l'invio della richiesta di reset password
Boundary	ResetPasswordView	(JSP) Interfaccia per l'impostazione della nuova password tramite token valido
Control	AutenticazioneController	Valida le credenziali dell'utente e gestisce l'avvio della sessione applicativa.
Control	RegistrazioneController	Gestisce la creazione di nuovi utenti verificando la validità del Codice Identificativo Speciale.
Control	RichiestaResetPasswordController	Avvia la procedura di recupero password generando un token di sicurezza temporaneo.
Control	ResetPasswordController	Consente l'impostazione di una nuova password previa validazione del token.
Control	LogoutController	Invalida la sessione utente terminando l'accesso al sistema.
Filter	AuthFilter	Protegge le risorse riservate verificando la presenza di una sessione valida; in caso contrario reindirizza alla pagina di login.
Entity	Utente	Rappresenta l'utente del sistema, includendo ruolo e dati anagrafici essenziali.
Entity	TokenResetPassword	Rappresenta il token temporaneo utilizzato per il recupero sicuro della password.
DAO	UserDAO	Gestisce la persistenza e il recupero delle informazioni dell'utente.
DAO	CredenzialiDAO	Gestisce la memorizzazione e la verifica delle credenziali di accesso (hash password con BCrypt).

Gestione Prenotazioni sale comuni

Questo sottosistema gestisce la prenotazione degli spazi comuni soggetti a utilizzo orario, come ad esempio lavanderia o sale condivise. Il flusso è di tipo sincrono e immediato: la prenotazione viene confermata solo se lo slot temporale richiesto risulta disponibile, senza necessità di approvazione da parte del Supervisore.

Categoria	Classe	Descrizione
Boundary	PrenotazioneView	(JSP) Visualizzazione dell'elenco delle prenotazioni effettuate dall'utente
Boundary	NuovaPrenotazioneView	(JSP) Form di inserimento di una nuova prenotazione oraria
Boundary	PrenotazioniSupervisoreView	(JSP) Vista riepilogativa delle prenotazioni lato Supervisore
Controller	PrenotazioneController	Gestisce la creazione, visualizzazione e cancellazione delle prenotazioni orarie. Verifica la disponibilità degli spazi ed evita situazioni di overbooking.

DAO	PrenotazioneDAO	Gestisce le operazioni di persistenza delle prenotazioni, verificando la disponibilità degli slot temporali richiesti e garantendo la coerenza dei dati.
Entity	Prenotazione	Rappresenta una prenotazione associata a uno spazio comune, comprensiva di data, orario e utente richiedente.

Gestione delle Risorse Condivise

Questo sottosistema gestisce l'utilizzo di beni condivisi del condominio (es. attrezzature, sale speciali), per i quali l'accesso è regolato tramite richieste soggette ad approvazione. Il flusso è di tipo asincrono, poiché richiede l'intervento del Supervisore per l'approvazione o il rifiuto delle richieste.

Categoria	Classe	Descrizione
Boundary	RisorseView	(JSP) Catalogo delle risorse condivise e invio richieste da parte
Boundary	RisorseSupervisoreView	(JSP) Gestione delle risorse e delle richieste pendenti lato Supervisore
Controller	RisorsaController	Permette ai Coinquilini di visualizzare il catalogo delle risorse disponibili e di inviare richieste di utilizzo per date specifiche.
Controller	SupervisoreRisorseController	Consente al Supervisore di inserire nuove risorse e di approvare o rifiutare le richieste pendenti.
DAO	RisorsaDAO	Gestisce la persistenza e il recupero delle informazioni relative alle risorse condivise disponibili.
DAO	RichiestaRisorsaDAO	Gestisce le richieste di utilizzo delle risorse. Il metodo esisteConflitto verifica la sovrapposizione temporale delle richieste.
Entity	Risorsa	Rappresenta una risorsa condivisa del sistema, con descrizione, regole di utilizzo ed eventuale penale.
Entity	RichiestaRisorsa	Rappresenta una richiesta di utilizzo di una risorsa per un determinato intervallo di date, con stato associato.

Gestione Ticket di Manutenzione

Il sottosistema di Gestione Ticket di Manutenzione consente ai Coinquilini di segnalare guasti o problematiche relative agli spazi e ai servizi comuni del condominio. Il sistema supporta la creazione delle segnalazioni, l'associazione di file allegati e la consultazione dello storico degli aggiornamenti inseriti nel corso della gestione del ticket.

Il Supervisore è responsabile della presa in carico delle segnalazioni, dell'aggiornamento dello stato del ticket e dell'inserimento di comunicazioni tecniche rivolte all'utente. Il flusso garantisce la tracciabilità completa delle operazioni e la persistenza coerente delle informazioni.

Categoria	Classe	Descrizione
-----------	--------	-------------

Boundary	TicketView (JSP)	(JSP) Visualizzazione dell'elenco dei ticket di manutenzione dell'utente
Boundary	NuovoTicketView	(JSP) Interfaccia per l'apertura di una nuova segnalazione
Boundary	DettagliTicketView	(JSP) Vista di dettaglio del ticket con storici aggiornamenti
Boundary	TicketSupervisoreView	(JSP) Elenco dei ticket lato Supervisore
Boundary	DettagliTicketSupervisoreView	(JSP) Vista di gestione del ticket (stato e aggiornamenti tecnici)
Boundary	DettagliTickeView	(JSP) Vista dettagli ticket per il coinquilino
Controller	TicketController	Gestisce la creazione del ticket da parte del Coinquilino e l'associazione degli eventuali file allegati
Controller	SupervisoreTicketController	Consente al Supervisore di aggiornare lo stato del ticket e inserire aggiornamenti tecnici nello storico
Entity	Ticket	Rappresenta la segnalazione di manutenzione con stato, priorità e dati descrittivi
Entity	AggiornamentoTicket	Rappresenta una voce dello storico del ticket, associata a un utente e a una data
DAO	TicketDAO	Gestisce la persistenza e il recupero dei ticket di manutenzione dal database
DAO	AllegatoDAO	Gestisce la persistenza delle informazioni relative ai file allegati e il collegamento con il ticket
DTO	AllegatoDTO	Contiene i metadati di un file allegato, permette il download sicuro

Gestione Tasse e Pagamenti

Il sottosistema di Gestione Tasse e Pagamenti supporta la gestione delle spese condominiali, consentendo al Supervisore di definire le tasse a carico dei Coinquilini e agli utenti di effettuare i relativi pagamenti. Il sistema traccia in modo coerente lo stato delle tasse, i pagamenti effettuati e le ricevute generate, garantendo la persistenza e la consultabilità delle informazioni economiche.

Il flusso applicativo prevede la creazione della tassa da parte del Supervisore, l'avvio del processo di pagamento da parte del Coinquilino e la successiva visualizzazione della ricevuta associata al pagamento effettuato.

Categoria	Classe	Descrizione
Boundary	SpeseView	(JSP) Visualizzazione delle tasse assegnate al Coinquilino
Boundary	PagamentoView	(JSP) Avvio del flusso di pagamento di una tassa
Boundary	RicevutaView	(JSP) Visualizzazione della ricevuta di pagamento
Boundary	TasseSupervisoreView	(JSP) Elenco e creazione delle tasse lato Supervisore
Boundary	DettagliTassaSupervisoreView	(JSP) Visualizzazione dei dettagli di una tassa lato Supervisore
Controller	SupervisoreTasseController	Consente al Supervisore di creare nuove tasse condominiali e assegnarle agli utenti

Controller	SpeseController	Gestisce l'avvio del flusso di pagamento da parte del Coinquilino
Controller	RicevutaController	Consente la visualizzazione delle ricevute di pagamento associate alle tasse saldate
Entity	TassaTrimestrale	Rappresenta una tassa condominiale con importo e scadenza
Entity	Pagamento	Rappresenta un'operazione di pagamento effettuata da un utente
Entity	Ricevuta	Documento che certifica l'avvenuto pagamento
DAO	TassaDAO	Gestisce la persistenza e il recupero delle informazioni relative alle tasse
DAO	PagamentoDAO	Gestisce la registrazione dei pagamenti e l'associazione con le ricevute

Gestione Eventi

Il sottosistema di Gestione Eventi supporta la bacheca degli eventi della comunità, consentendo ai Coinquilini di consultare gli eventi disponibili e gestire la propria partecipazione (iscrizione/disiscrizione), nel rispetto dei vincoli applicativi (posti disponibili, evento futuro, prevenzione iscrizioni duplicate).

Il Supervisore può invece gestire gli eventi pubblicati, includendo la creazione e la cancellazione. Il controller di bacheca coordina il recupero dei dati e l'esposizione alla view, mentre i controller dedicati gestiscono le azioni puntuali di iscrizione/disiscrizione e le operazioni amministrative del Supervisore.

Categoria	Classe	Descrizione
Boundary	BachecaEventiView	(JSP) Visualizzazione della bacheca eventi per il Coinquilino
Boundary	ConfermaIscrizioneEventoView	(JSP) Interfaccia di conferma dell'iscrizione ad un evento selezionato
Boundary	EventiSupervisoreView	(JSP) Elenco degli eventi gestiti dal Supervisore
Boundary	NuovoEventoView	(JSP) Creazione di un nuovo evento da parte del Supervisore
Controller	BachecaEventiController	Recupera e prepara i dati della bacheca eventi per l'utente autenticato, includendo lo stato di partecipazione e la disponibilità posti
Controller	ConfermaIscrizioneEventoController	Gestisce la visualizzazione della pagina di conferma e l'azione di iscrizione, intercettando condizioni di errore (es. duplicata)
Controller	DisiscrizioneEventoController	Gestisce la rimozione della partecipazione dell'utente da un evento a cui era iscritto
Controller	SupervisoreEventiController	Permette al Supervisore la gestione degli eventi (creazione e cancellazione, oltre alle operazioni previste dal prototipo)
DTO	EventoBachecaDTO	Oggetto di trasferimento per la rappresentazione

		in bacheca (dati evento + stato utente: iscritto/non iscritto, posti, ecc.)
Entity	Evento	Rappresenta un evento della comunità con informazioni principali (titolo, descrizione, luogo, data, posti)
DAO	EventoDAO	Gestisce il recupero degli eventi per la bacheca e le operazioni di persistenza/partecipazione (iscrizione, disiscrizione, eliminazione, ecc.)

Classi DAO e Persistenza dei Dati

In questa sezione vengono riepilogate tutte le classi di accesso ai dati (Data Access Object) implementate nel sistema OikoNaos, evidenziando per ciascuna la tabella del database gestita e la responsabilità principale all'interno del livello di persistenza.

Classe DAO	Tabella DB gestita	Responsabilità principale
UserDAO	utente	Gestisce la creazione degli utenti e l'aggiornamento delle credenziali a seguito di reset password
CredenzialiDAO	credenziali	Gestisce l'autenticazione sicura degli utenti e la verifica degli hash delle password
TokenResetPasswordDAO	tokenresetpassword	Gestisce la creazione, validazione e invalidazione dei token temporanei per il recupero password
CodiceIdentificativoDAO	codiceidentificativo	Gestisce la validazione e l'invalidazione dei Codici Identificativi Speciali per la registrazione
PrenotazioneDAO	prenotazione	Gestisce le prenotazioni degli spazi comuni a fasce orarie, verificando la disponibilità degli slot
RisorsaDAO	risorsacondivisa	Gestisce la persistenza e il recupero delle risorse condivise disponibili
RichiestaRisorsaDAO	richiestarisorsa	Gestisce le richieste di utilizzo delle risorse e la verifica di conflitti temporali
TicketDAO	ticket	Gestisce la creazione, il recupero e il filtraggio dei ticket di manutenzione
AggiornamentoTicketDAO	aggiornamentoticket	Gestisce lo storico degli aggiornamenti e delle comunicazioni associate ai ticket
AllegatoDAO	allegato	Gestisce i metadati dei file allegati ai ticket di manutenzione
TassaDAO	tassatrimestrale	Gestisce la creazione e il recupero delle tasse condominiali
PagamentoDAO	pagamento	Gestisce la registrazione dei pagamenti e

		l'associazione alle tasse
RicevutaDAO	ricevuta	Gestisce la generazione e il recupero delle ricevute di pagamento
EventoDAO	evento / partecipazione_evento	Gestisce la persistenza degli eventi, le iscrizioni, le disiscrizioni e la cancellazione degli eventi

Data Transfer Objects (DTO)

In questa sezione sono descritte le classi Data Transfer Object utilizzate nel sistema OikoNaos per il trasferimento dei dati tra il livello di applicazione (Controller) e il livello di presentazione (View – JSP). L'impiego dei DTO consente di disaccoppiare il modello di persistenza dall'interfaccia utente, evitando l'esposizione diretta delle entità del dominio e permettendo l'aggregazione di informazioni provenienti da più sorgenti. I DTO non contengono logica di business, ma esclusivamente dati strutturati necessari alla corretta visualizzazione delle informazioni lato utente.

Classe	Descrizione e ruolo nel sistema
EventoBachecaDTO	Trasporta le informazioni di un evento destinate alla visualizzazione nella bacheca, includendo i dati principali dell'evento e lo stato di partecipazione dell'utente corrente (iscritto / non iscritto), nonché informazioni utili alla gestione delle azioni disponibili
AllegatoDTO	Contiene i metadati di un file allegato a un ticket di manutenzione, consentendo la visualizzazione e il download sicuro del contenuto senza esporre direttamente il file system